

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Lab 9 - WEP40 Wireless Packet Decryption and Aircrack Forensics

Lab Title:	WEP40 Wireless Packet Decryption and Aircrack Forensics
Student Name:	Yusuf Oluwaseun Dada
Student ID:	2025/FWSD/11339
Course Code:	SBT-DF203
Course Name:	Basic Networking Skills for Digital Forensics

Instructor: Mr. Aminu Idris, AMCPN

Date: 25/09/2026

1. Lab Scenario and Learning Outcomes

A historical wireless CTF capture protected by deprecated WEP40 has been supplied. I will have to preserve and decompress the evidence, identify IEEE 802.11 traffic, recover or validate the 40-bit key using Aircrack-ng, decrypt the capture offline, inventory network endpoints, carve transferred files, and explain why WEP is cryptographically unsafe.

Outcomes:

- ✓ To identify management, control, and data frames in an 802.11 capture.
- ✓ Explain WEP40 structure, RC4 keystream use, IV reuse, and integrity limitations.
- ✓ Use Aircrack-ng on a supplied historical capture only.
- ✓ Decrypt WEP traffic using airdecap-ng and verify output files.
- ✓ Extract IP/MAC endpoints, protocols, images, and HTML from decrypted traffic.
- ✓ Distinguish a WEP key from a human-readable passphrase, and document optional CTF password analysis.
- ✓ Recommend modern wireless security controls.

2. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux VM	Offline capture analysis.
Wireless tools	aircrack-ng suite	Recover/validate key and decrypt capture.
Packet tools	Wireshark and tshark	Inspect encrypted/decrypted frames.
Carving tools	foremost, binwalk, file, strings	Recover transferred objects.
Compression	xz-utils	Decompress file.xz.
Evidence	Historical CodeGate CTF capture	Authorized offline dataset.

3. Lab Folder Setup and Evidence

```
(kali@kali)-[~/Desktop]
$ mkdir -p ~/SBT-DF203-Lab9/{evidence,working,exported,reports,screenshots,scripts}
$ cd ~/SBT-DF203-Lab9
$ pwd
/home/kali/SBT-DF203-Lab9
$ find . -maxdepth 1 -type d -print
./
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ wget -O evidence/file.xz \
  'https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz'
--2026-09-24 18:51:41-- https://raw.githubusercontent.com/ctfs/write-ups-2015/master/codegate-ctf-2015/programming/good-crypto/file.xz
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12837660 (12M) [application/octet-stream]
Saving to: 'evidence/file.xz'

evidence/file.xz      100%[=====>] 12.24M  546KB/s  in 19s

2026-09-24 18:52:01 (677 KB/s) - 'evidence/file.xz' saved [12837660/12837660]
```

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ sha256sum evidence/file.xz | tee reports/file_xz_sha256.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  evidence/file.xz
```

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ cp --preserve=timestamps evidence/file.xz working/file_working.xz
$ unxz -k working/file_working.xz
$ file working/file_working
working/file_working: pcap capture file, microsecond ts (little-endian) - version 2.4 (802.11, capture length 65535)
$ sha256sum working/file_working.xz working/file_working | tee reports/working_hashes.txt
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d  working/file_working
```

The Lab functions?

It installs the aircrack-ng suite (for key recovery/decryption), Wireshark/TShark (for inspection), and carving tools (foremost/binwalk) needed later. wget downloads the historical CTF capture, compressed as .xz, into evidence/ and untouched original. Hashing it immediately preserves proof of the original download. cp makes a working copy; unxz -k decompresses it while keeping (-k) the original .xz intact, producing the actual capture file working/file_working. file identifies the decompressed file's actual type (confirming it's a real packet capture, not something else), and the final sha256sum records both the compressed and decompressed working files.

4. Chain-of-Custody Worksheet

Field	Your Entry
Case/lab identifier	SBT-DF203-Lab9
Trainee name	Yusuf Oluwaseun Dada
Date and time started	23/09/2026 4:30pm
Evidence file name(s)	file.xz (original), file_working.xz / file_working (working copies)
Source or generation method	Downloaded from public CTF write-up repository (CodeGate CTF 2015)
Original SHA-256	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6
Working-copy SHA-256	file_working.xz: dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6 (matches original, confirms clean copy); file_working (decompressed): c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d

5. Part A - Inventory the Wireless Capture

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ CAP=working/file_working
capinfos "$CAP" | tee reports/capinfos.txt
tshark -r "$CAP" -q -z io,phs | tee reports/protocol_hierarchy.txt
tshark -r "$CAP" -Y "wlan" -T fields \
  -e frame.number -e frame.time -e wlan.fc.type -e wlan.fc.subtype -e wlan.sa -e wlan.da -e wlan.bssid \
  | head -n 100 | tee reports/wlan_frame_sample.tsv
File name:          working/file_working
File type:          Wireshark/tcpdump/... - pcap
File encapsulation: IEEE 802.11 Wireless LAN
File timestamp precision: microseconds (6)
Packet size limit:  file hdr: 65535 bytes
Number of packets:  45 k
File size:          14 MB
Data size:          13 MB
Capture duration:   272.969795 seconds
Earliest packet time: 2015-03-06 02:51:54.364525
Latest packet time:  2015-03-06 02:56:27.334320
Data byte rate:     49 kBps
Data bit rate:      395 kbps
Average packet size: 299.13 bytes
Average packet rate: 165 packets/s
SHA256:             c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d
SHA1:               cb929fd6e407c13639e3168b0f35f921788478d3
Strict time order:   False
Number of interfaces in file: 1
Interface #0 info:
  Encapsulation = IEEE 802.11 Wireless LAN (20 - ieee-802-11)
  Capture length = 65535
  Time precision = microseconds (6)
  Time ticks per second = 1000000
  Number of stat entries = 0
```

Output Explanation:

capinfos summarizes the file (packet count, duration, encapsulation type, confirming it's an 802.11 capture). -z io,phs prints TShark's protocol hierarchy statistics, showing every protocol layer present in the file and what percentage of packets use each, as a fast way to see at a glance whether WEP, IP, TCP, HTTP etc. appear at all. The wlan filter extracts 802.11-specific fields: frame type/subtype (which frame category it belongs to), source/destination MAC, and BSSID (the access point's MAC address identifying the network). head -n 100 keeps the sample manageable to review by hand.

Frame Category	Type Value	Purpose	Example Evidence
Management	0	Association, authentication, beacons	Frame 54: type 0, subtype 8 (Beacon), sent by 00:26:66:55:97:d6, broadcast to ff:ff:ff:ff:ff:ff
Control	1	Flow control and acknowledgements	Frames 1–53, 55+ (the overwhelming majority of your sample): type 1, subtype 11 (RTS - Request to Send); frame 47: subtype 13 (ACK)
Data	2	Carries network payload	Not present in the first 100 sampled frames, but confirmed present overall, protocol hierarchy output shows 15,713 data frames total out of 45,169

6. Part B - Identify WEP Protection and Key Parameters

1. Confirm that protected data frames are present in the capture.
2. Record the BSSID, source/destination MAC addresses, and IV-related fields where TShark dissects them.
3. Explain that 64-bit WEP normally combines a 24-bit Initialization Vector (IV) with a 40-bit secret key.
4. Explain why repeated IVs and RC4 keystream weaknesses make WEP unsuitable for any real use today.

```
(kali@kali) ~[SBT-DF203-Lab9]
$ tshark -r "$CAP" -Y "wlan.fc.protected==1" -T fields \
  -e frame.number -e frame.time_epoch -e wlan.sa -e wlan.da -e wlan.bssid -e wlan.wep.iv -e wlan.wep.key \
  | head -n 200 | tee reports/wep_protected_frames.tsv

tshark -r "$CAP" -Y "wlan.fc.protected==1" -T fields -e wlan.wep.iv \
  | sort | uniq -c | sort -nr | head -n 30 | tee reports/repeated_iv_summary.txt
9432  1425628319.232982000  00:26:66:55:97:d4  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x67ffec  0
10130 1425628320.138252000  f0:f6:1c:68:96:7c  00:26:66:55:97:d6  00:26:66:55:97:d6  0xc2cbeb  0
10155 1425628320.176656000  f0:f6:1c:68:96:7c  04:8d:38:48:e5:b4  00:26:66:55:97:d6  0xc3cbeb  0
10157 1425628320.177168000  f0:f6:1c:68:96:7c  04:8d:38:48:a8:b5  00:26:66:55:97:d6  0xc4cbeb  0
10159 1425628320.177168000  f0:f6:1c:68:96:7c  08:10:77:92:7c:2f  00:26:66:55:97:d6  0xc5cbeb  0
10161 1425628320.177168000  f0:f6:1c:68:96:7c  00:26:66:55:97:d4  00:26:66:55:97:d6  0xc6cbeb  0
10163 1425628320.177168000  f0:f6:1c:68:96:7c  c8:3a:35:58:c4:ba  00:26:66:55:97:d6  0xc7cbeb  0
10165 1425628320.177167000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xc8cbeb  0
10167 1425628320.177686000  00:26:66:55:97:d4  f0:f6:1c:68:96:7c  00:26:66:55:97:d6  0x68ffec  0
10170 1425628320.180760000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x69ffec  0
10171 1425628320.182288000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0xc9cbeb  0
10172 1425628320.187417000  00:26:66:55:97:d4  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6affec  0
10173 1425628320.187408000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0xc9cbeb  0
10175 1425628320.187919000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcacbeb  0
10177 1425628320.188952000  f0:f6:1c:68:96:7c  33:33:00:00:00:02  00:26:66:55:97:d6  0x6bffec  0
10178 1425628320.189976000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6cffec  0
10183 1425628320.192016000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcbcbeb  0
10185 1425628320.193048000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6dffec  0
10224 1425628320.259088000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0xcccbeb  0
10228 1425628320.262680000  f0:f6:1c:68:96:7c  ff:ff:ff:ff:ff:ff  00:26:66:55:97:d6  0x6effec  0
10229 1425628320.262678000  00:26:66:55:97:d4  f0:f6:1c:68:96:7c  00:26:66:55:97:d6  0x6fffec  0
10233 1425628320.263184000  f0:f6:1c:68:96:7c  00:26:66:55:97:d4  00:26:66:55:97:d6  0xcdcbeb  0
10261 1425628320.419350000  f0:f6:1c:68:96:7c  00:26:66:55:97:d4  00:26:66:55:97:d6  0xc9cbeb  0
10266 1425628320.420374000  f0:f6:1c:68:96:7c  01:00:5e:00:00:02  00:26:66:55:97:d6  0xcfcbeb  0
```

Output Explanation:

wlan.fc.protected==1 isolates frames with the WEP/encryption bit set in their frame control header, these are the encrypted data frames. Extracting wlan.wep.iv pulls out each frame's 24-bit Initialization Vector, a value meant to be combined with the secret key to produce a unique RC4 keystream per packet. The second command counts how often each IV value repeats across the capture (sort | uniq -c | sort -nr), because the IV space is only 24 bits (about 16.7 million possibilities), which a busy network will reuse IVs relatively quickly, and repeated IVs mean repeated keystreams, which is exactly the weakness that makes WEP statistically breakable.

Item	Your Finding
BSSID observed	00:26:66:55:97:d6 (consistent across the beacon frame and every WEP-protected frame, which is the access point)
Source/destination MACs observed	Two primary stations: 00:26:66:55:97:d4 and f0:f6:1c:68:96:7c, communicating through the AP (00:26:66:55:97:d6); also many multicast/broadcast destinations (ff:ff:ff:ff:ff:ff, 01:00:5e:..., 33:33:...) consistent with normal LAN discovery/multicast traffic
Number of protected data frames	all data frames in a WEP network are typically protected which is 200

Item	Your Finding
Most-repeated IV value(s) and count	0x42e8eb and 0x28d3eb, each appearing 7 times; 0x95dfeb appearing 6 times, several dozen other IVs repeat 3-5 times each

7. Part C - Recover or Validate the WEP40 Key

```
(kali@kali)-[~/SBT-DF203-Lab9]
$ aircrack-ng "$CAP" | tee reports/aircrack_output.txt
# Expected historical training key from the slide: A4:3D:F6:F3:74
printf 'A4:3D:F6:F3:74\n' | tee reports/validated_wep40_key_masked.txt
Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

# BSSID          ESSID          Encryption
1 00:26:66:55:97:D6  cgnetwork      WEP (15477 IVs)

Choosing first network as target.

Reading packets, please wait ...
Opening working/file_working
Read 45169 packets.

1 potential targets

Attack will be restarted every 5000 captured ivs.

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB  depth  byte(vote)
0   0/ 1    A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) B7(19200) 26(18944) 68(18944)
1   0/ 1    3D(23040) 51(20736) 07(20480) 62(19968) 78(19968) 1F(19712) B0(19712) BD(19456) 80(19200) 85(19200)
2   0/ 1    F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944) 4F(18944) A8(18944)
3   1/ 10   F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 09(19456) 20(19456) 8F(19200)

Aircrack-ng 1.7

[00:00:00] Tested 83 keys (got 15477 IVs)

KB  depth  byte(vote)
0   0/ 1    A4(22784) 62(20992) A8(19968) B6(19968) 42(19456) 6E(19456) 91(19200) B7(19200) 26(18944) 68(18944)
1   0/ 1    3D(23040) 51(20736) 07(20480) 62(19968) 78(19968) 1F(19712) B0(19712) BD(19456) 80(19200) 85(19200)
2   0/ 1    F6(23808) E4(20992) D0(20736) 68(20224) 95(19712) 38(19456) 0C(19200) 45(18944) 4F(18944) A8(18944)
3   1/ 10   F3(20480) C5(19968) D0(19968) 3E(19712) 43(19456) 52(19456) B2(19456) 09(19456) 20(19456) 8F(19200)
4   6/ 9    01(19712) 20(19456) 3E(19456) 52(19456) 5C(19456) 90(19456) F9(19456) 45(18944) 85(18944) 95(18944)

KEY FOUND! [ A4:3D:F6:F3:74 ]
Decrypted correctly: 100%

A4:3D:F6:F3:74
```

The aircrack-ng output showing the selected BSSID, IV count, and recovered/matching key.

aircrack-ng analyzes the captured IVs statistically, insufficient IV space plus RC4's key-scheduling flaws to recover the WEP key without needing to guess it by brute force.

8. Part D - Decrypt the Capture Offline

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ cd working
airdecap-ng -w A43DF6F374 file_working | tee ../reports/airdecap_output.txt
cd ..
find working -maxdepth 1 -type f -ls | tee reports/decrypted_files_inventory.txt
sha256sum working/* | tee reports/all_working_file_hashes.txt
Total number of stations seen      10
Total number of packets read      45169
Total number of WEP data packets  15477
Total number of WPA data packets  0
Number of plaintext data packets  0
Number of decrypted WEP packets   15477
Number of corrupted WEP packets   0
Number of decrypted WPA packets   0
Number of bad TKIP (WPA) packets  0
Number of bad CCMP (WPA) packets  0
3571786 13904 -rw-rw-r-- 1 kali kali 14234002 Sep 24 18:52 working/file_working
3571876 12488 -rw-rw-r-- 1 kali kali 12784501 Sep 24 19:08 working/file_working-dec
3571781 12540 -rw-rw-r-- 1 kali kali 12837660 Sep 24 18:52 working/file_working.xz
c17a3f9b955e84f5befd476dbd55c67286d1e3eea9ab402d5359cac0874ebb2d working/file_working
167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04 working/file_working-dec
dd54144caef34f228bf4b87a9101ca7173969376f7ed357bd068061d4f4b8d6 working/file_working.xz
```

airdecap-ng -w takes the WEP key with colons removed (A43DF6F374, matching the A4:3D:F6:F3:74 and decrypts the entire capture using it, producing a new, unencrypted file, typically named with a -dec suffix. find -ls to lists every file now present in working/ in order to identify this new decrypted file by name and size. Hashing everything in the folder gives me a complete, timestamped record of every working file at this stage of the investigation.

9. Part E - Extract Endpoints and Conversations

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ DEC=$(find working -maxdepth 1 -type f -name '*-dec*' | head -n 1)
echo "Decrypted capture: $DEC" | tee reports/decrypted_capture_path.txt
Decrypted capture: working/file_working-dec
```

```
(kali㉿kali)-[~/SBT-DF203-Lab9]
$ DEC=$(find working -maxdepth 1 -type f -name '*-dec*' | head -n 1)
echo "Decrypted capture: $DEC" | tee reports/decrypted_capture_path.txt
Decrypted capture: working/file_working-dec

(kali㉿kali)-[~/SBT-DF203-Lab9]
$ tshark -r "$DEC" -q -z endpoints,eth | tee reports/ethernet_endpoints.txt
tshark -r "$DEC" -q -z endpoints,ip | tee reports/ip_endpoints.txt
tshark -r "$DEC" -q -z conv,tcp | tee reports/tcp_conversations.txt

tshark -r "$DEC" -Y "ip" -T fields -e frame.number -e wlan.sa -e wlan.da -e ip.src -e ip.dst -e _ws.col.Protocol \
| head -n 200 | tee reports/ip_mac_mapping_sample.tsv
```

Ethernet Endpoints							
Filter:<No Filter>							
	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	
Apple_68:96:7c	15407	12 MB	7076	616 kB	8331	11 MB	
EFMNetworks_55:97:d4	15020	12 MB	8178	11 MB	6842	593 kB	
ASUSTekCOMPU_2a:c2:7a	358	281 kB	223	270 kB	135	11 kB	
IPv4mcast_fb	42	6,648 bytes	0	0 bytes	42	6,648 bytes	
IPv6mcast_fb	36	7,022 bytes	0	0 bytes	36	7,022 bytes	
Broadcast	31	3,386 bytes	0	0 bytes	31	3,386 bytes	
IPv4mcast_7f:ff:fa	31	11 kB	0	0 bytes	31	11 kB	
IPv6mcast_16	10	920 bytes	0	0 bytes	10	920 bytes	
IPv6mcast_02	9	630 bytes	0	0 bytes	9	630 bytes	
IPv4mcast_16	3	180 bytes	0	0 bytes	3	180 bytes	
IPv4mcast_02	2	92 bytes	0	0 bytes	2	92 bytes	
NetisTechnol_48:e5:b4	1	42 bytes	0	0 bytes	1	42 bytes	
NetisTechnol_48:a8:b5	1	42 bytes	0	0 bytes	1	42 bytes	
08:10:77:92:7c:2f	1	42 bytes	0	0 bytes	1	42 bytes	
TendaTechnol_58:c4:ba	1	42 bytes	0	0 bytes	1	42 bytes	

The \$(find ...) command automatically locates whatever file aircrack-ng actually created, not to have to hardcode its exact name. -z endpoints,eth and -z endpoints,ip summarize every unique Ethernet and IP address seen in the decrypted traffic, with packet/byte totals for each. -z conv,tcp summarizes every TCP conversation. The final command maps each IP-layer packet back to its underlying 802.11 MAC addresses (wlan.sa/wlan.da) alongside the IP addresses and protocol, bridging the wireless (Layer 2) and network (Layer 3) views of the same traffic.

10. Part F - Extract Images, HTML and Other Objects

1. First try Wireshark's File > Export Objects for HTTP (or other recognized protocols) — this is the cleanest extraction method when the protocol is properly reconstructed.
2. Use TShark's object export for HTTP where available, as an automatable equivalent to the GUI method.
3. Use foremost as a carving method, and document that carved files may include false positives or lack their original filenames — carving works on raw byte signatures, not protocol structure, so it's a fallback, not a first choice.
4. Open any recovered objects only inside the lab VM, and hash them before viewing — treat recovered files as potentially untrusted content, same as any other evidence.

```
(kali@kali)-[~/SBT-DF203-Lab9]
└─$ aircrack-ng "$CAP" | tee reports/aircrack_output.txt
# Expected historical training key from the slide: A4:3D:F6:F3:74
printf 'A4:3D:F6:F3:74\n' | tee reports/validated_wep40_key_masked.txt
Reading packets, please wait...
Opening working/file_working
Read 45169 packets.

# BSSID          ESSID          Encryption
1 00:26:66:55:97:D6 cgnetwork      WEP (15477 IVs)

Choosing first network as target.

Reading packets, please wait...
Opening working/file_working
```

11. Part G - Optional Historical Passphrase Challenge (Instructor-Assessed)

12. WEP Security Analysis

- i. WEP uses RC4 with a short 24-bit IV, allowing IV reuse in busy networks, as directly observed in Part B's repeated_iv_summary.txt.
- ii. The same keystream reuse leaks relationships between plaintexts and enables statistical key recovery, exactly what aircrack-ng exploited in Part C during validation WEP40 key

- iii. WEP provides weak integrity and no modern protection against replay or key-recovery attacks.
- iv. The recommended replacement is WPA2-AES/CCMP or WPA3, with strong authentication and protected management frames where supported.
- v. Legacy WEP should be removed entirely rather than strengthened with a longer passphrase, the protocol's cryptographic design is fundamentally broken, not just weakly configured.

13. Required Findings Worksheet

Question	Finding
	198.90.20.111 and 199.27.79.193 (largest by volume, both port 80/HTTP), plus Apple-infrastructure IPs (17.172.239.x) over TLS; DNS resolution via 8.8.8.8; local gateway 192.168.0.1 (DHCP)
Key protocols observed	TCP (139 instances in sample), TLSv1/TLSv1.2 (18 combined, HTTPS traffic), DNS (14), HTTP (11, plaintext, potentially recoverable in Part F), MDNS (8, Apple service discovery), DHCP (6), IGMPv2 (4)
Recovered images/HTML	
Optional router passphrase	Protected appendix only
Security conclusion	WEP's 24-bit IV space is too small for any busy network to avoid reuse; combined with RC4's related-key weaknesses, this allowed statistical key recovery, directly demonstrated by successful decryption revealing extensive plaintext-visible activity, including DNS lookups, unencrypted HTTP sessions, and device identification (Apple MDNS/DHCP) that should never have been exposed. WEP should be replaced entirely with WPA2-AES/CCMP or WPA3.

14. Executive Summary

This report documents the forensic decryption and analysis of a historical WEP40-protected wireless capture (file.xz, SHA-256: dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6) conducted as part of SBT-DF203 Lab 9.

Inventory of the capture (Part A) identified 45,169 total frames across management, control, and data categories, associated with BSSID 00:26:66:55:97:D6 (ESSID: cgnetwork). Analysis of protected data frames (Part B) confirmed [exact protected-frame count pending] WEP-encrypted frames, with numerous repeated IV values observed (e.g. 0x42e8eb and 0x28d3eb, each appearing 7 times) - consistent with WEP's known 24-bit IV-space limitation.

Using Aircrack-ng (Part C), the WEP40 key was recovered/validated as A4:3D:F6:F3:74, working against 15,477 unique IVs captured for the target network. The capture was subsequently decrypted offline using airdecap-ng (Part D), producing working/file_working-dec (SHA-256: 167c91994c269777f9048227deb89882caf3cf3c763977f2059604f9a6a40b04), confirmed valid by the appearance of readable ARP, DHCP, IP/TCP, DNS, HTTP, and TLS traffic when opened directly in Wireshark.

Endpoint analysis (Part E) identified host 192.168.0.15 (MAC f0:f6:1c:68:96:7c, an Apple device) communicating primarily with remote web servers 198.90.20.111 and 199.27.79.193 over HTTP, alongside Apple-infrastructure hosts (17.172.239.x) over TLS, DNS resolution via 8.8.8.8, and local network services (DHCP, MDNS) via gateway 192.168.0.1.

This exercise directly demonstrates why WEP is unsuitable for any current use: its short IV space and RC4 keystream reuse allowed statistical key recovery well within the scope of a training exercise, exposing all subsequent traffic, including DNS lookups, unencrypted HTTP sessions, and device-identifying broadcast/discovery traffic, to full decryption. Modern networks should use WPA2-AES/CCMP or WPA3 with protected management frames.

Key limitations of this analysis are: this is a historical, artificially-constrained CTF dataset with a small, tractable IV count; real-world WEP key recovery on a low-traffic network may require substantially more captured IVs and correspondingly more time.